MINI PROJECT REPORT

IMDB MOVIE RATING SCRAPER

NAME:M.MONIKA

CLASS:III-B.SC.IT

COLLEGE:VIVEKANANDHA COLLEGE OF ARTS AND SCIENCE FOR WOMEN(AUTONOMOUS)

BATCH:B1

MODE:ONLINE

Abstract

The Movie Rating Scraper is a Python-based mini project developed to collect movie information from The Movie Database (TMDB) API. The application retrieves details such as movie title, release date, average rating, vote count, overview, and original language.

The project uses the Requests library to communicate with the TMDB API and the Pandas library to organize the collected information into a structured table. The results are displayed in the Python terminal and saved in a CSV file named movie_ratings.csv.

The application searches for ten popular movies and demonstrates how API integration can simplify the collection and organization of online movie information. It also provides practical knowledge of Python programming, JSON processing, exception handling, and data export.

1. Introduction

Movies are an important form of entertainment, and people often compare films based on their ratings, release dates, and descriptions. Searching for information about multiple movies manually can take considerable time.

Python provides libraries that make it possible to retrieve information from online services and process it automatically. An API, or Application Programming Interface, allows one software application to request information from another application.

In this project, the TMDB API is used to search for movies and retrieve their available details. The collected information is processed using Python and stored in CSV format for convenient access.

1.1 Problem Statement

Manually searching for movie information and recording ratings can be time-consuming. There is a need for a simple application that can retrieve details about multiple movies, display them systematically, and save the results for future use.

1.2 Proposed Solution

The proposed system uses Python to send requests to the TMDB API, extract the required movie details, and save the collected records in a CSV file. It includes basic error handling for network problems and unsuccessful requests.

2. Project Description

The Movie Rating Scraper is a command-line application developed using Python. It searches for ten predefined movies using the TMDB search endpoint.

The program sends each movie name to the API and receives a response containing matching movie records. It selects the first result returned by the API and extracts the available information.

The retrieved data is stored in a Pandas DataFrame and exported to a CSV file. Users can view the results in the terminal or open the CSV file using Microsoft Excel.

2.1 Main Features

- Searches for ten predefined movies.
- Retrieves movie titles and release dates.
- Collects TMDB average ratings and vote counts.
- Retrieves movie descriptions and original language codes.
- Handles common network and API request errors.
- Displays the collected information in the terminal.
- Exports the data to a CSV file.

2.2 Objectives

The primary objectives are:

- To develop a movie information collection application using Python.
- To retrieve movie details through the TMDB API.
- To understand HTTP requests and API responses.
- To process JSON data using Python.
- To organize movie details using Pandas.
- To save collected information in CSV format.
- To implement basic error handling.
- To gain practical experience in Python application development.

2.3 Scope of the Project

The project focuses on retrieving information for a predefined list of movies. It is suitable for beginners who want to learn API integration and data processing. The application currently works through the terminal and does not include a graphical user interface, a database, or automatic live monitoring.

3. Technologies Used

3.1 Python

Python is the primary programming language used to implement the application. It supports API communication, data processing, functions, and exception handling.

3.2 TMDB API

The Movie Database API provides movie information through HTTP endpoints. The project uses the movie search endpoint to find matching records.

3.3 Requests Library

The Requests library sends HTTP GET requests to the API and receives responses. It also supports request timeouts and HTTP error handling.

3.4 Pandas Library

Pandas is used to store the collected movie information in a DataFrame and export it to a CSV file.

3.5 JSON

JSON is the response format used to transfer structured movie information from the API to the Python application.

3.6 Visual Studio Code

Visual Studio Code is used to create, edit, execute, and debug the Python source code.

3.7 CSV File Format

CSV stands for Comma-Separated Values. It stores tabular data in a format that can be opened using spreadsheet applications

3.1 Hardware Requirements

- Computer or laptop
- At least 4 GB RAM recommended
- Internet connection
- Sufficient storage space for Python and project files

3.2 Software Requirements

- Windows, Linux, or macOS
- Python 3
- Visual Studio Code
- Requests library
- Pandas library
- Valid TMDB API credentials

3.3 Installation Commands

- Install the required libraries using the VS Code terminal:
- python -m pip install requests pandas
- The user must also obtain a TMDB API key and configure it before running the application.

4. System Architecture

The application follows a simple request-and-response architecture.
Architecture diagram:
User / Python Program
↓
Movie List
↓
TMDB API Request
↓
TMDB API Server
↓
JSON Response
↓
Data Extraction and Error Handling
↓
Pandas DataFrame
↓
Terminal Output and CSV File

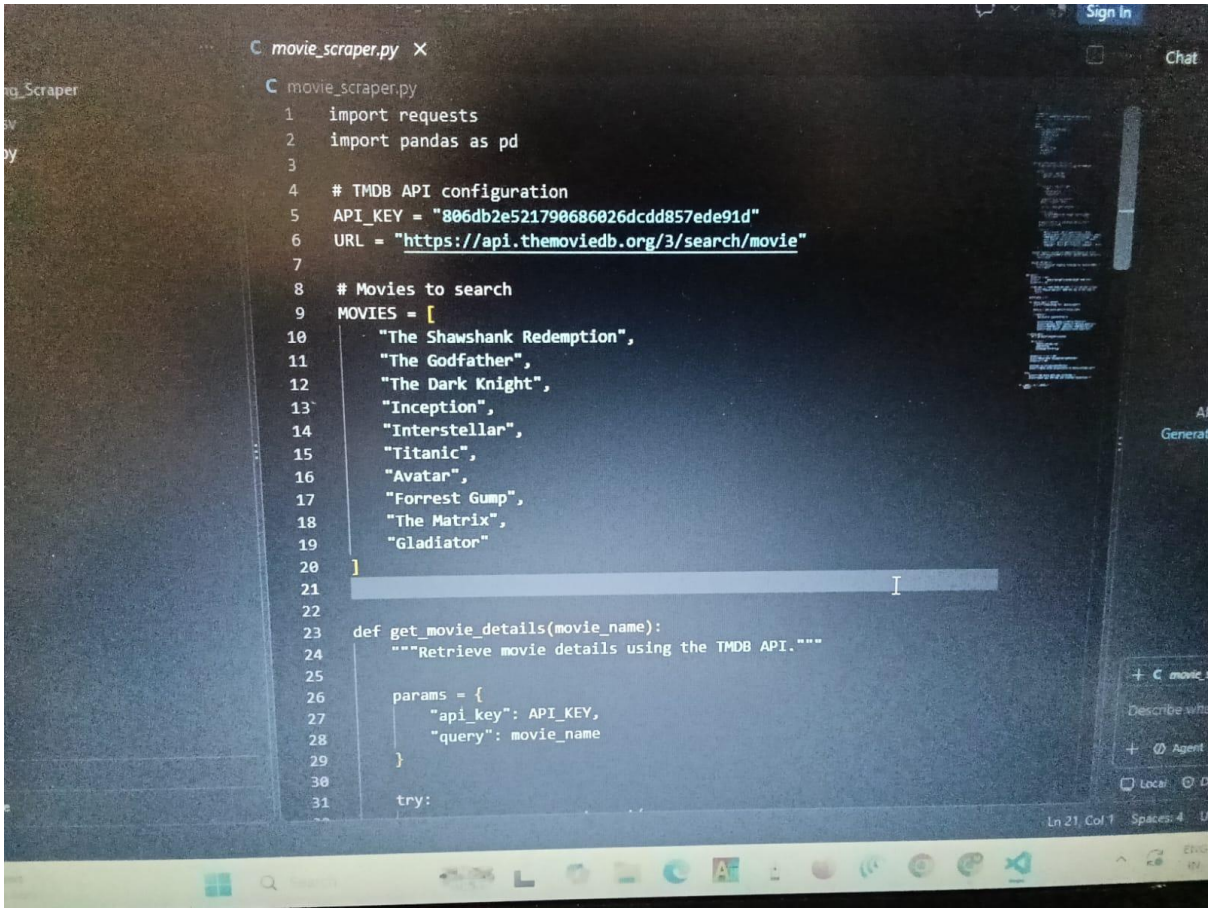
4.1 Working Procedure

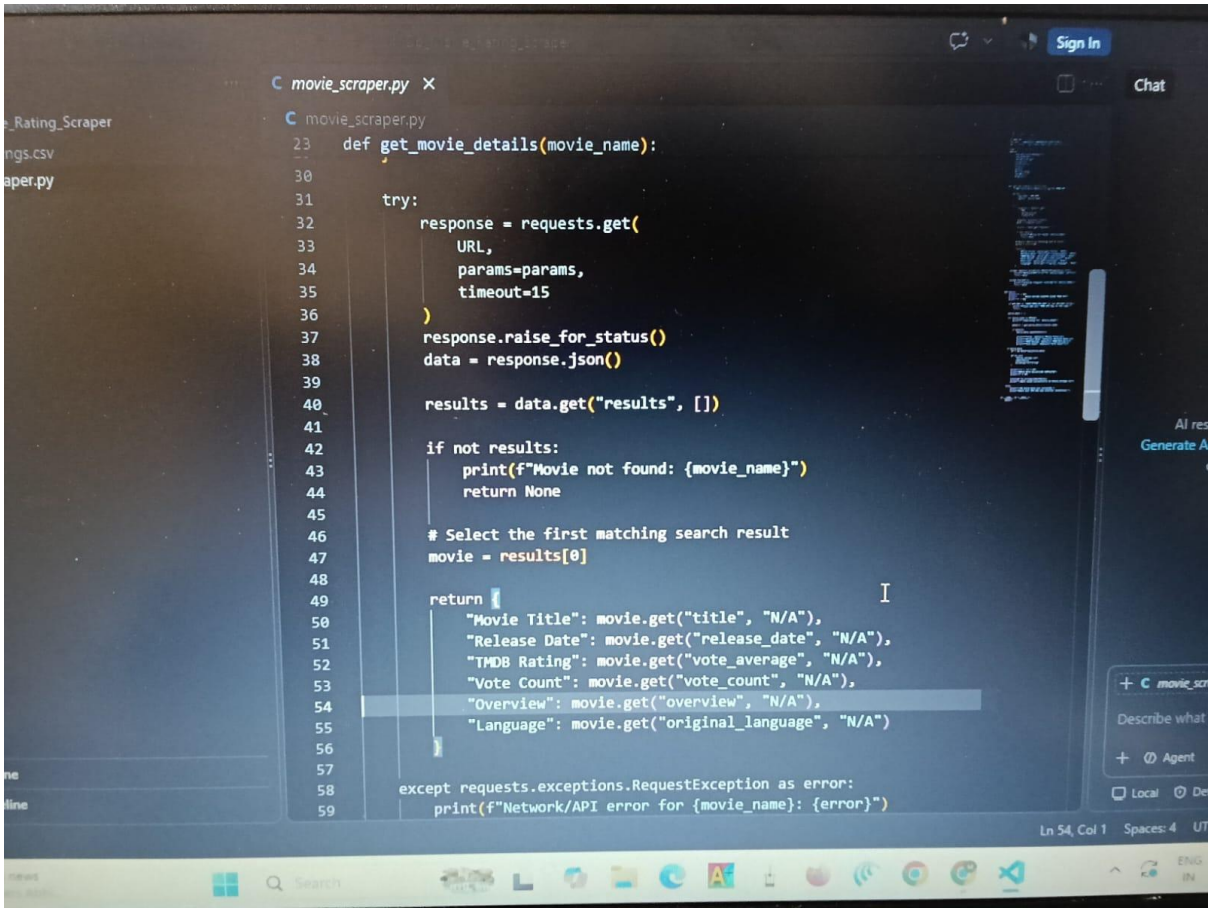
- Step 1: API Configuration
  The program defines the TMDB API key and the movie search URL.
- Step 2: Movie Selection
  A predefined list contains ten movie titles to be searched.
- Step 3: Sending Requests
  The requests.get() method sends an HTTP GET request for each movie name.
- Step 4: Receiving Responses
  The API returns data in JSON format. The program checks the HTTP response and extracts the list of search results.
- Step 5: Extracting Movie Details
  The application retrieves the title, release date, average rating, vote count, overview, and original language.
- Step 6: Organizing Data
  The retrieved movie records are collected in a Python list and converted into a Pandas DataFrame.
- Step 7: Saving Results
  The DataFrame is saved to movie_ratings.csv using UTF-8 encoding.
- Step 8: Displaying Results
  The program displays the collected records and a completion message in the terminal.

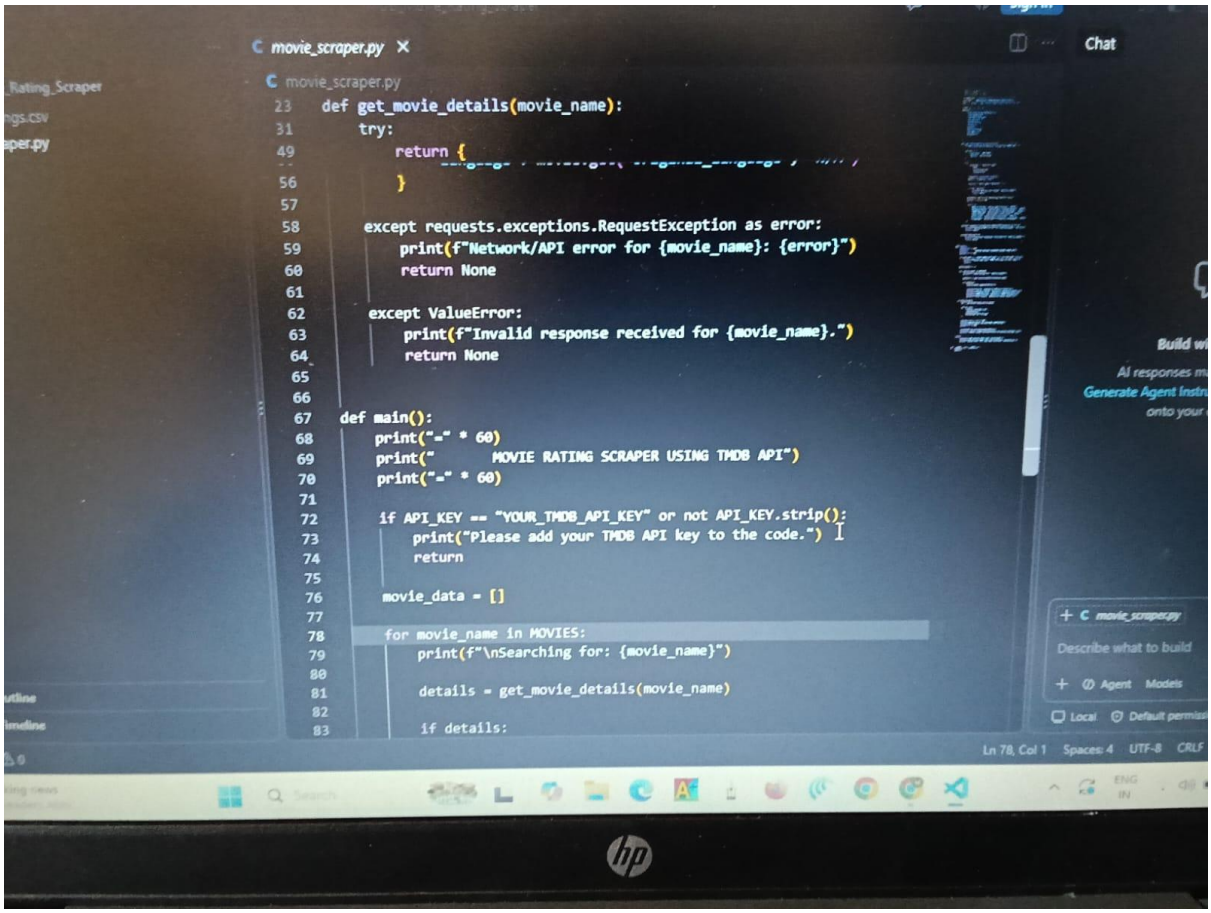
4.2 Methodology

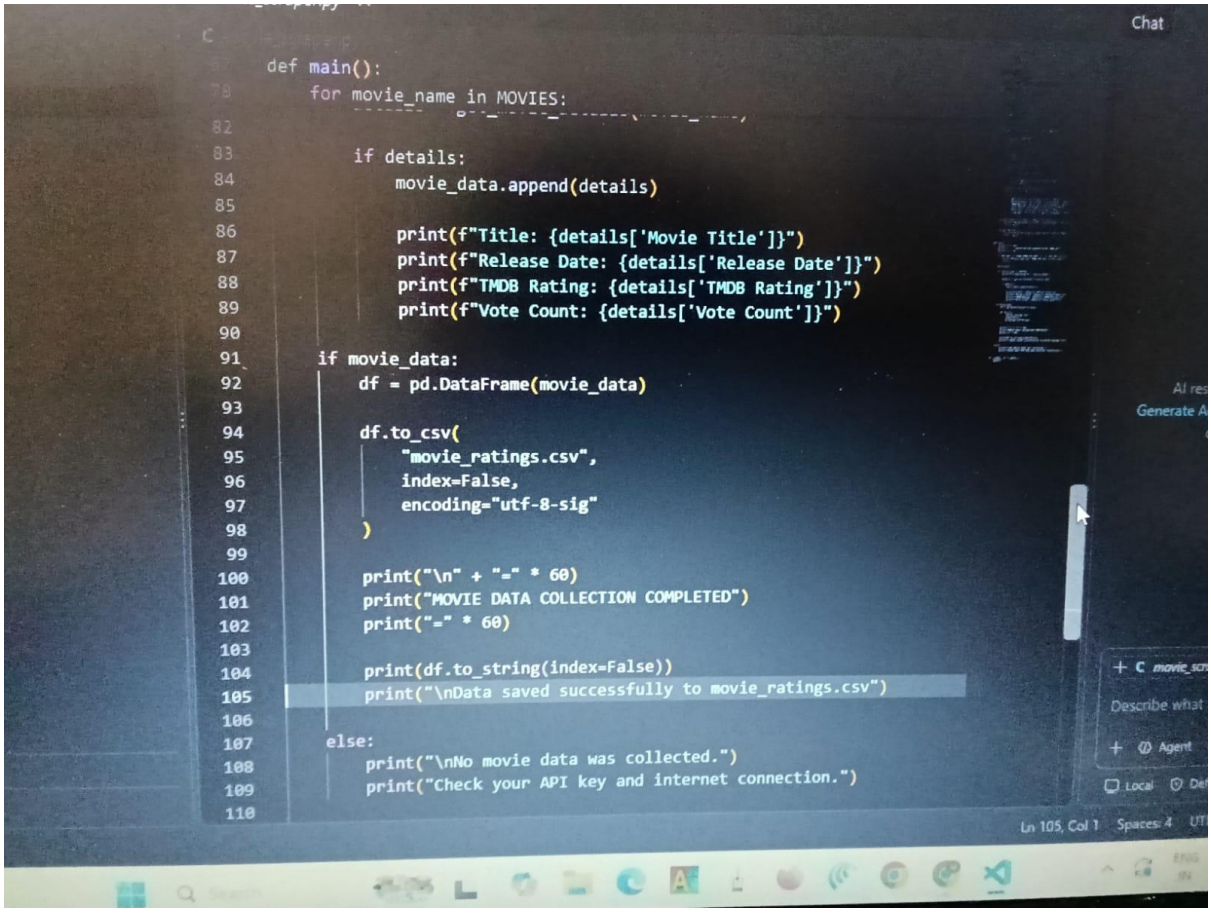
- The project uses API-based data retrieval rather than direct HTML web scraping. This approach allows the program to access structured information provided by TMDB.
- The application performs repeated searches, extracts selected fields, handles common request errors, and exports successful records. Its output depends on the data returned by TMDB at the time of execution.

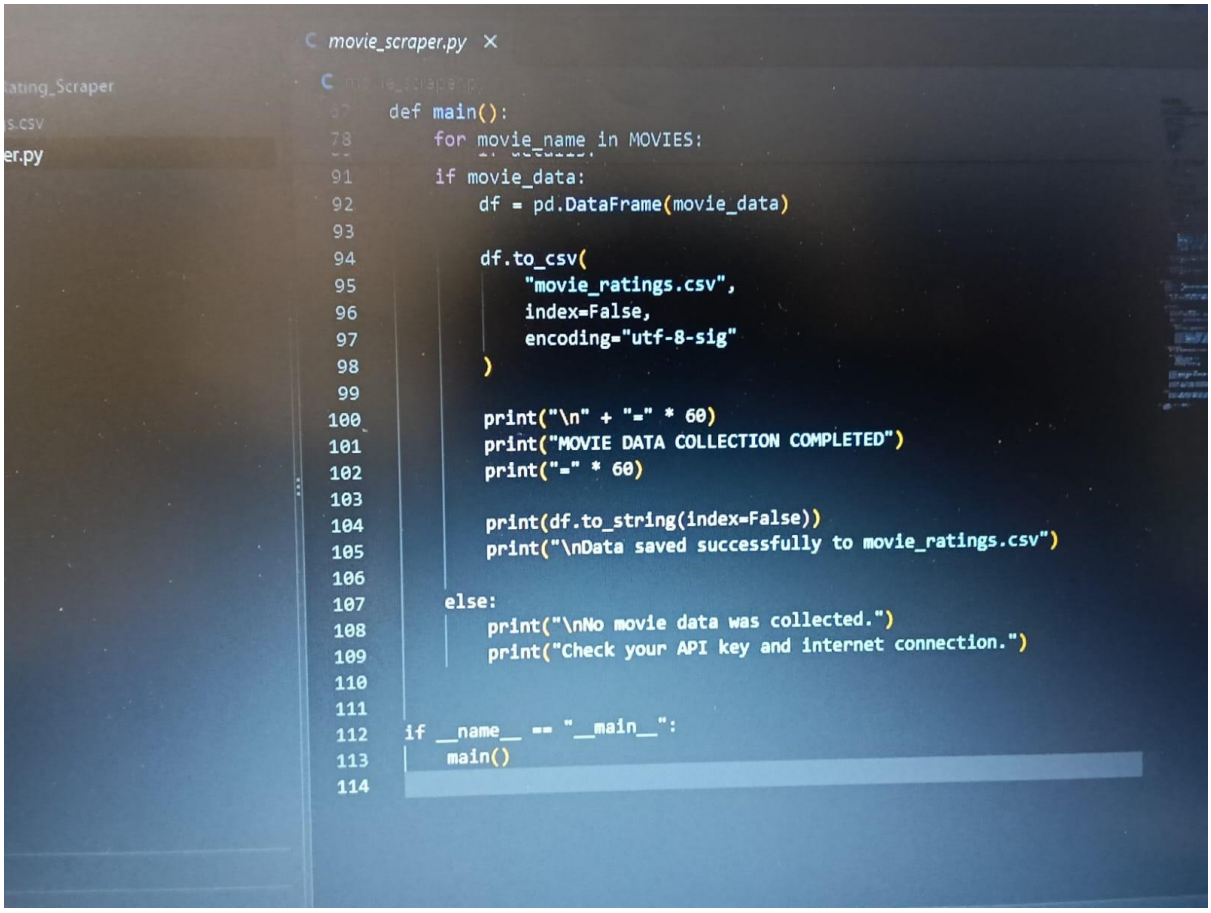
5.SOURCE CODE:



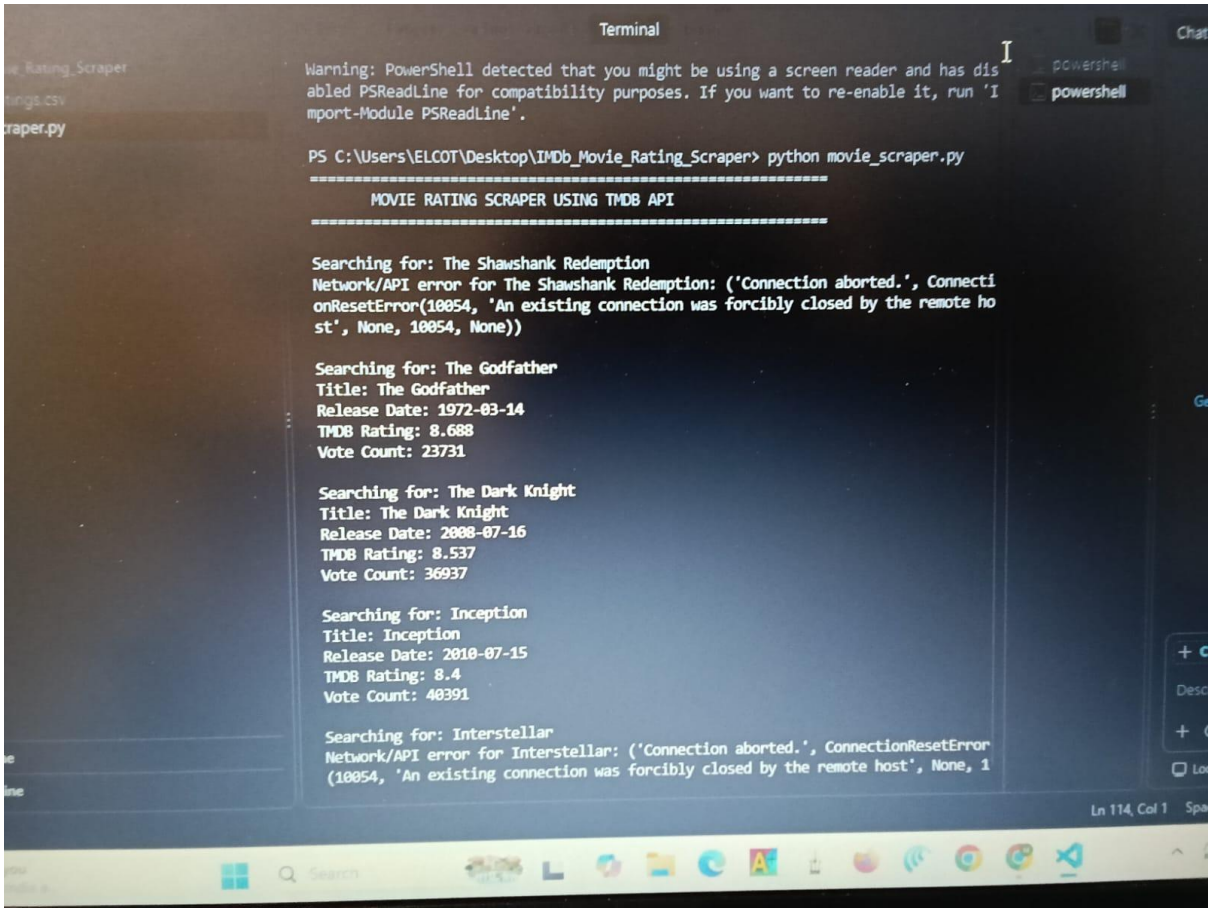

movie_scraper.py
Sign In
Chat
def get_movie_details(movie_name):
try:
response = requests.get(
URL,
params=params,
timeout=15
)
response.raise_for_status()
data = response.json()
results = data.get("results", [])
if not results:
print(f"Movie not found: {movie_name}")
return None
Select the first matching search result
movie = results[0]
return {
"Movie Title": movie.get("title", "N/A"),
"Release Date": movie.get("release_date", "N/A"),
"TMDB Rating": movie.get("vote_average", "N/A"),
"Vote Count": movie.get("vote_count", "N/A"),
"Overview": movie.get("overview", "N/A"),
"Language": movie.get("original_language", "N/A")
}
except requests.exceptions.RequestException as error:
print(f"Network/API error for {movie_name}: {error}")
Ln 54, Col 1
Spaces: 4

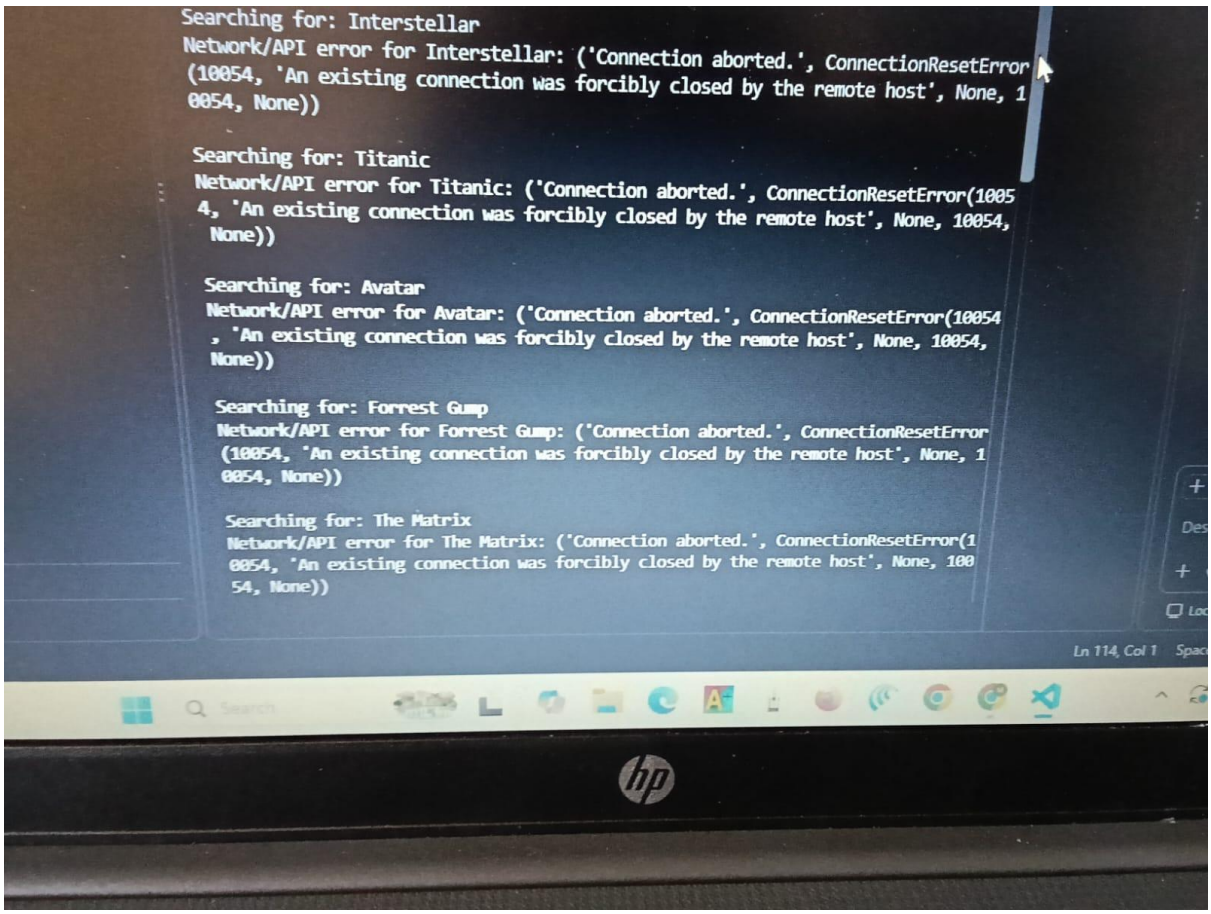

movie_scraper.py
Chat
def get_movie_details(movie_name):
try:
return {
}
except requests.exceptions.RequestException as error:
print(f"Network/API error for {movie_name}: {error}")
return None
except ValueError:
print(f"Invalid response received for {movie_name}.")
return None
def main():
print("=" * 60)
print("MOVIE RATING SCRAPER USING TMDB API")
print("=" * 60)
if API_KEY == "YOUR_TMDB_API_KEY" or not API_KEY.strip():
print("Please add your TMDB API key to the code.")
return
movie_data = []
for movie_name in MOVIES:
print(f"\nSearching for: {movie_name}")
details = get_movie_details(movie_name)
if details:
Ln 78, Col 1
Spaces: 4
UTF-8
CRLF

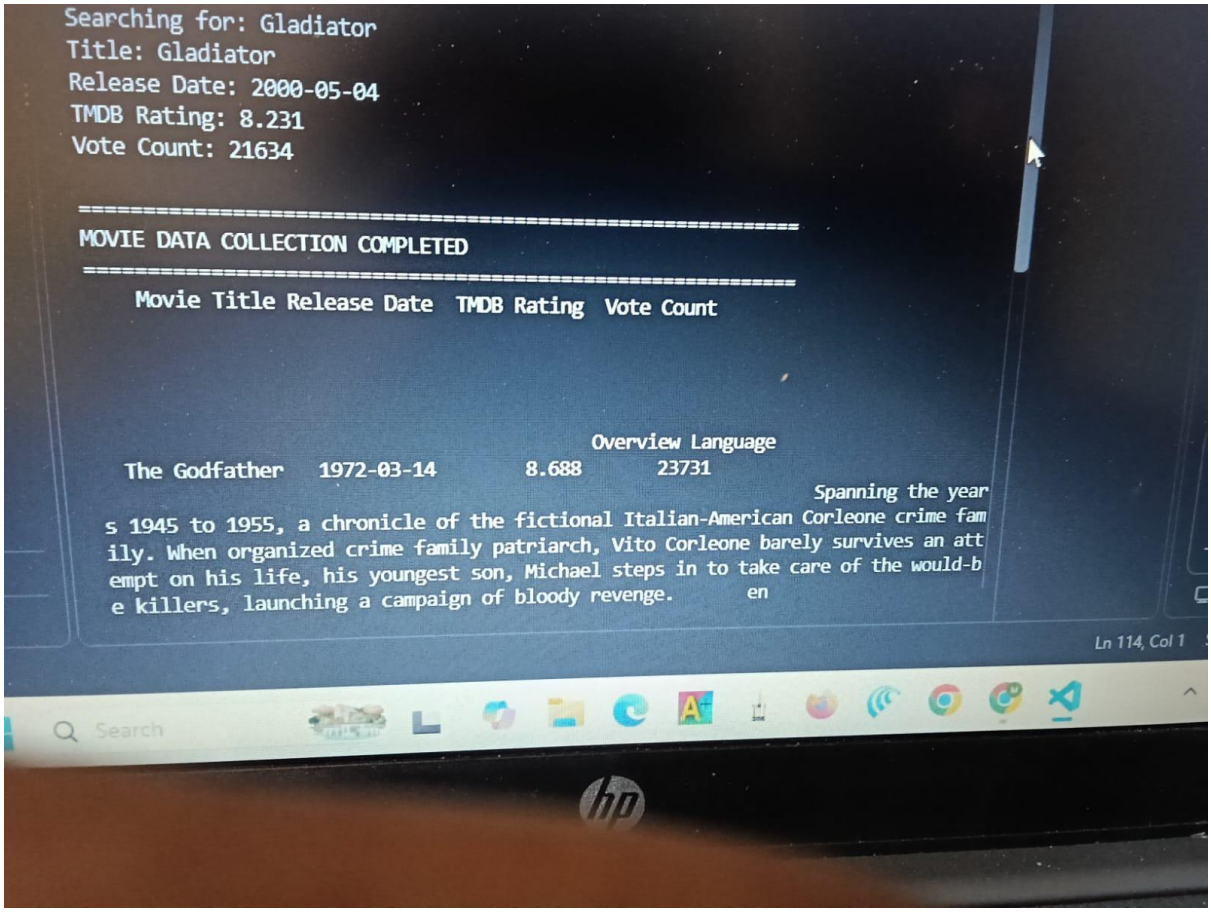

Chat
def main():
for movie_name in MOVIES:
if details:
movie_data.append(details)
print(f"Title: {details['Movie Title']}")
print(f"Release Date: {details['Release Date']}")
print(f"TMDB Rating: {details['TMDB Rating']}")
print(f"Vote Count: {details['Vote Count']}")
if movie_data:
df = pd.DataFrame(movie_data)
df.to_csv(
"movie_ratings.csv",
index=False,
encoding="utf-8-sig"
)
print("\n" + "=" * 60)
print("MOVIE DATA COLLECTION COMPLETED")
print("=" * 60)
print(df.to_string(index=False))
print("\nData saved successfully to movie_ratings.csv")
else:
print("\nNo movie data was collected.")
print("Check your API key and internet connection.")
Describe what
Agent
Ln 105, Col 1
Spaces: 4

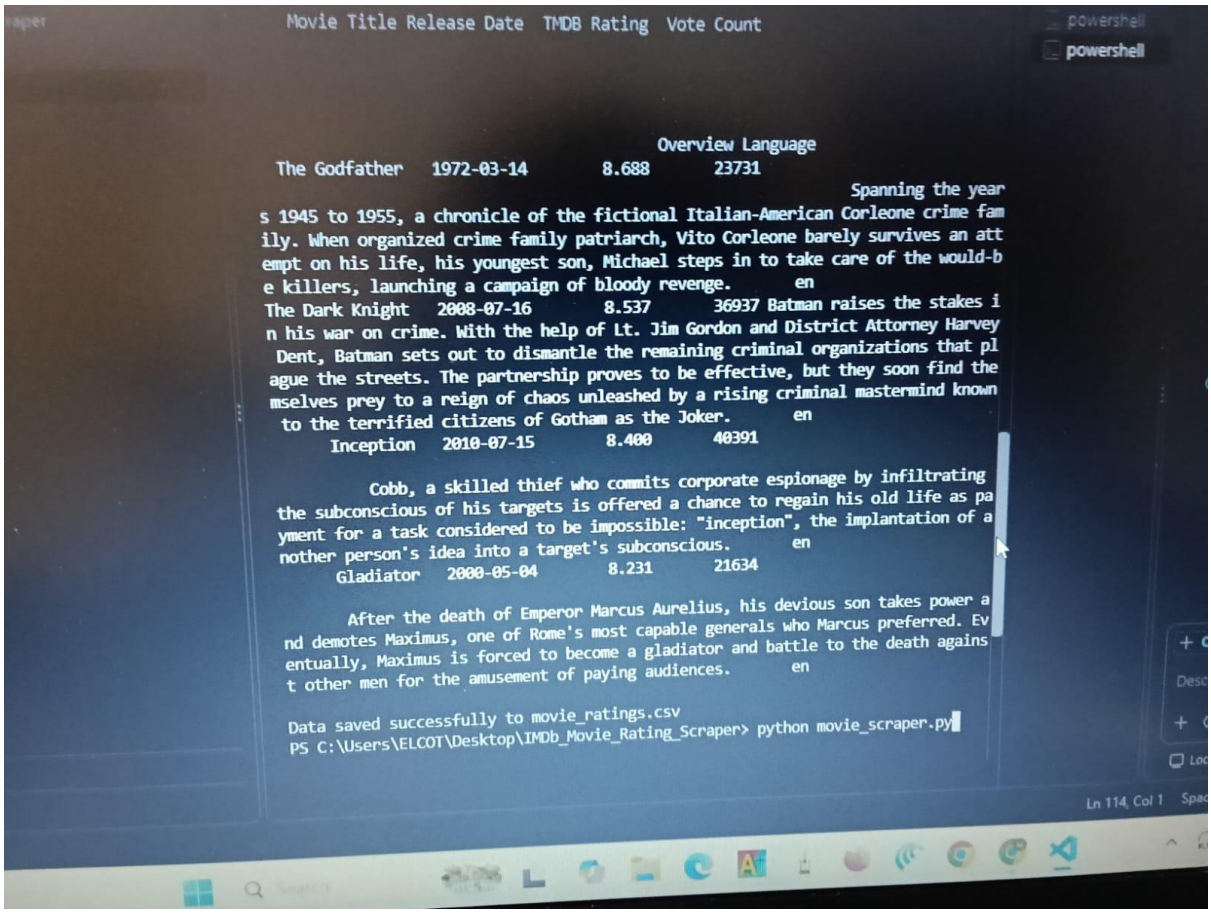

movie_scraper.py
def main():
for movie_name in MOVIES:
if movie_data:
df = pd.DataFrame(movie_data)
df.to_csv(
"movie_ratings.csv",
index=False,
encoding="utf-8-sig"
)
print("\n" + "=" * 60)
print("MOVIE DATA COLLECTION COMPLETED")
print("=" * 60)
print(df.to_string(index=False))
print("\nData saved successfully to movie_ratings.csv")
else:
print("\nNo movie data was collected.")
print("Check your API key and internet connection.")
if __name__ == "__main__":
main()

6.OUTPUT:


Terminal
Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.
PS C:\Users\ELCOT\Desktop\IMDb_Movie_Rating_Scraper> python movie_scraper.py
MOVIE RATING SCRAPER USING TMDB API
Searching for: The Shawshank Redemption
Network/API error for The Shawshank Redemption: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Searching for: The Godfather
Title: The Godfather
Release Date: 1972-03-14
TMDB Rating: 8.688
Vote Count: 23731
Searching for: The Dark Knight
Title: The Dark Knight
Release Date: 2008-07-16
TMDB Rating: 8.537
Vote Count: 36937
Searching for: Inception
Title: Inception
Release Date: 2010-07-15
TMDB Rating: 8.4
Vote Count: 40391
Searching for: Interstellar
Network/API error for Interstellar: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 1
powershell
Ln 114, Col 1


Searching for: Interstellar
Network/API error for Interstellar: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Searching for: Titanic
Network/API error for Titanic: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Searching for: Avatar
Network/API error for Avatar: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Searching for: Forrest Gump
Network/API error for Forrest Gump: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Searching for: The Matrix
Network/API error for The Matrix: ('Connection aborted.', ConnectionResetError(10054, 'An existing connection was forcibly closed by the remote host', None, 10054, None))
Ln 114, Col 1


Searching for: Gladiator
Title: Gladiator
Release Date: 2000-05-04
TMDB Rating: 8.231
Vote Count: 21634
MOVIE DATA COLLECTION COMPLETED
Movie Title Release Date TMDB Rating Vote Count
Overview Language
The Godfather 1972-03-14 8.688 23731
Spanning the years 1945 to 1955, a chronicle of the fictional Italian-American Corleone crime family. When organized crime family patriarch, Vito Corleone barely survives an attempt on his life, his youngest son, Michael steps in to take care of the would-be killers, launching a campaign of bloody revenge. en
Ln 114, Col 1
Search


Movie Title Release Date TMDB Rating Vote Count
powershell
Overview Language
The Godfather 1972-03-14 8.688 23731
Spanning the years 1945 to 1955, a chronicle of the fictional Italian-American Corleone crime family. When organized crime family patriarch, Vito Corleone barely survives an attempt on his life, his youngest son, Michael steps in to take care of the would-be killers, launching a campaign of bloody revenge. en
The Dark Knight 2008-07-16 8.537 36937 Batman raises the stakes in his war on crime. With the help of Lt. Jim Gordon and District Attorney Harvey Dent, Batman sets out to dismantle the remaining criminal organizations that plague the streets. The partnership proves to be effective, but they soon find themselves prey to a reign of chaos unleashed by a rising criminal mastermind known to the terrified citizens of Gotham as the Joker. en
Inception 2010-07-15 8.400 40391
Cobb, a skilled thief who commits corporate espionage by infiltrating the subconscious of his targets is offered a chance to regain his old life as payment for a task considered to be impossible: "inception", the implantation of another person's idea into a target's subconscious. en
Gladiator 2000-05-04 8.231 21634
After the death of Emperor Marcus Aurelius, his devious son takes power and demotes Maximus, one of Rome's most capable generals who Marcus preferred. Eventually, Maximus is forced to become a gladiator and battle to the death against other men for the amusement of paying audiences. en
Data saved successfully to movie_ratings.csv
PS C:\Users\ELCOT\Desktop\IMDb_Movie_Rating_Scraper> python movie_scraper.py
Ln 114, Col 1

6.3 Results

The project is designed to collect details for ten predefined movies and store successfully retrieved records in a CSV file.

The main expected result is a structured dataset containing movie details and ratings. The actual number of records collected should be confirmed after execution because a search may return no results or encounter an API error.

7. Testing

Testing is performed to verify whether the application works correctly under different conditions.

- Test case - Expected result
- Valid API key - Movie searches proceed
- Invalid API key - API error is reported
- Movie not found - A message is displayed and processing continues
- Internet unavailable - Request error is handled
- Successful movie search - Available movie details are extracted
- CSV export - Retrieved records are saved in a CSV file
- No successful records - No-data message is displayed

7.1 Testing Outcome

- The test results must be recorded after running the application. The expected behaviour should be compared with the actual terminal output and generated CSV file.
- A successful test should confirm that the application retrieves the expected information, handles errors appropriately, and saves the available data.

7.2 Project Outcomes

- The project demonstrates the following outcomes:
- Successful integration of Python with an external API, when valid credentials and connectivity are available.
- Retrieval of structured movie information from JSON responses.

- Organization of records using Pandas.
- Export of movie information to a CSV file.
- Basic handling of network and HTTP request errors.
- Practical understanding of API-based data collection

8. Conclusion

- The Movie Rating Scraper Using Python and TMDB API is a simple application for retrieving and organizing movie information. It uses the Requests library to communicate with TMDB, processes JSON responses, and uses Pandas to save the results in CSV format.
- The project provides practical experience in Python programming, API integration, exception handling, and data processing. It also demonstrates how repetitive data collection tasks can be automated using a small Python application.
- The project can serve as a foundation for more advanced movie information systems

8.1 Future Development

The following enhancements can be implemented in the future:

- Add a search interface where users can enter movie titles.
- Improve movie selection by matching titles and release years.
- Retrieve genres, directors, cast, and additional movie details when supported by the API.
- Add a graphical user interface using Tkinter or a web interface using Flask.
- Store movie details in a database such as SQLite.
- Add filtering and sorting based on ratings and release dates.
- Generate charts to compare movie ratings.
- Allow users to export the results to Excel.
- Add better logging, retry handling, and API error messages

8.2 References

Python Documentation — https://docs.python.org/3/

Requests Documentation — https://requests.readthedocs.io/

Pandas Documentation — https://pandas.pydata.org/docs/

TMDB API Documentation — https://developer.themoviedb.org/docs